

Degree-Mass Message Passing for Betweenness Ranking in Directed and Undirected Networks

Anonymous Author(s)

Abstract

Computing the importance of nodes in networks is a long-standing fundamental problem that has driven extensive study of various centrality measures. A particularly well-known centrality measure is betweenness centrality, whose exact computation becomes prohibitive on large-scale networks. Graph Neural Network (GNN) models have thus been proposed to predict the ranking of nodes by betweenness centrality. However, existing GNN-based methods either have graph-size-dependent parameter counts or are limited to undirected graphs. We propose a lightweight GNN architecture that exploits the empirically observed relationship between betweenness centrality and multi-hop degree mass. This motivates the use of degree masses as size-invariant node features. To improve generalization, we train on synthetic graphs whose degree distributions more closely match those of real-world networks, including directed and undirected scale-free graphs and uniformly directed hyperbolic random graphs. We evaluate our model on 14 real-world networks spanning eight domains, including social, email, and citation networks, across both directed and undirected regimes. The experiments show that our model improves the Kendall τ_b correlation by up to 24.6% on undirected and 10.9% on directed graphs, while using 56× fewer parameters than the lightest competing GNN baseline and achieving competitive inference time, with up to a 24.5× speedup on selected directed graphs.

CCS Concepts

• **Computing methodologies** → *Machine learning; Learning to rank; Neural networks; Learning paradigms; Supervised learning; Machine learning approaches*; • **Mathematics of computing** → *Discrete mathematics*.

Keywords

graph neural network, betweenness centrality, large-scale graph analysis, hyperbolic graph model, degree mass

ACM Reference Format:

Anonymous Author(s). 2026. Degree-Mass Message Passing for Betweenness Ranking in Directed and Undirected Networks. In *Proceedings of 35th International ACM Conference on Information and Knowledge Management (CIKM '26)*. ACM, New York, NY, USA, 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CIKM '26, Rome, Italy

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/26/08
<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Identifying important nodes in networks is a crucial task in many applications. For instance, in social networks, influential individuals can significantly impact information dissemination, while in email communication networks, key individuals often act as hubs for information exchange and organizational coordination [25, 26]. Centrality measures formalize this concept. In particular, betweenness centrality is effective in identifying nodes that act as bridges or bottlenecks within a network, as it quantifies the importance of a node based on its position in the network by measuring the fraction of shortest paths between pairs of nodes that pass through a given node [13].

Despite its utility, computing betweenness centrality exactly is computationally expensive: Brandes' algorithm runs in $O(|V||E|)$ time, making it impractical for large-scale real-world applications [7]. This has motivated the investigation of approximation methods, including sampling-based algorithms, parallel and distributed computing, and, more recently, learning-based methods such as Graph Neural Networks. In many downstream applications, the exact centrality value is less important than the induced ranking of nodes, making betweenness approximation naturally suited to learning-to-rank formulations. Existing GNN-based methods, however, still face important limitations. Some methods rely on node-specific embeddings or padded graph representations, causing the number of learnable parameters to depend on the input graph size. Other parameter-efficient architectures are designed primarily for undirected graphs and do not directly model the asymmetric flow of information in directed networks. This leaves a gap for a compact, size-invariant architecture that can approximate betweenness rankings across both directed and undirected graphs. We propose BRAVA-GNN (Betweenness Ranking Approximation via Degree Mass GNN)¹, a lightweight architecture for betweenness-centrality ranking. It is designed to handle both directed and undirected graphs, while enhancing accuracy and computational efficiency and maintaining a parameter count independent of graph size. In particular:

- We propose BRAVA-GNN, a compact message-passing architecture for betweenness-centrality ranking that uses multi-hop degree-mass features instead of node-specific embeddings. This makes the number of learnable parameters independent of the input graph size and reduces the parameter count by 56× compared to the lightest GNN baseline.
- We introduce a synthetic training strategy based on directed and undirected scale-free graphs and uniformly directed hyperbolic random graphs (UDHY), with generation parameters chosen from empirical real-world network statistics. Our ablations show that this mixed training distribution improves generalization across graph domains.

¹Brava means "good" or "excellent" in Italian, alluding to the model's performance.

- We evaluate BRAVA-GNN on 14 real-world networks spanning eight domains and compare it against learning-based and sampling-based baselines. BRAVA-GNN is best on all test graphs. Against the strongest baseline, its gains reach 24.6% on undirected and 10.9% on directed graphs. It remains competitive in inference time and reaches up to a 24.5× speedup on selected directed graphs.

The source code is available at <https://anonymous.4open.science/r/BRAVA-GNN-A0BD>.

2 Related Work

The exact computation of betweenness centrality relies on Brandes' algorithm [7], which becomes prohibitively expensive for large-scale graphs due to its time complexity of $O(|V||E|)$ for unweighted networks and $O(|V||E| + |V|^2 \log(|V|))$ for weighted networks. Under the Strong Exponential Time Hypothesis [22], no truly subquadratic algorithm exists for computing betweenness centrality, as this would imply a subquadratic solver for k-SAT* [5]. Consequently, there is considerable interest in distributed computing approaches and approximation algorithms.

2.1 Unsupervised Approximation Algorithms

Geisberger et al. [15] improve pivot-sampling approximations of betweenness centrality by rescaling the dependency contributions obtained from randomly selected source nodes. Their distance-aware linear and bisection scaling schemes reduce the overestimation of nodes close to sampled pivots and improve empirical accuracy over the Brandes and Pich [8] estimator. Riondato and Kornaropoulos [36] propose a sampling-based approach that leverages the concept of VC-dimension to determine the number of shortest paths required to achieve a desired level of accuracy. Borassi and Natale [6] propose KADABRA, which combines balanced bidirectional BFS with adaptive sampling to approximate betweenness centrality in directed and undirected graphs. For fixed additive-error approximation, they report average speedups of more than 100× over previous approximation algorithms on graphs with approximately 10,000 nodes. Pellegrina and Vandin [35] propose SILVAN, a progressive sampling algorithm that uses Monte Carlo Empirical Rademacher Averages with a novel variance-dependent partitioning scheme to obtain sharper non-uniform bounds on the betweenness centrality estimates. Mahmoody et al. [29] address the centrality maximization problem via a randomized algorithm based on shortest-path sampling that provides theoretical approximation guarantees. Yoshida [43] uses hypergraph sketches to compute adaptive betweenness centrality, updating scores in almost linear time as nodes are selected. Regarding centrality variants, Zhang et al. [44] combine deterministic graph traversals with random walk sampling to approximate spanning centrality, while Brunelli et al. [9] introduce polynomial-time algorithms for computing betweenness in temporal graphs under various path optimality and waiting constraints.

2.2 Parallel and Distributed Algorithms

AlGhamdi et al. [1] compute the exact betweenness centrality by implementing a parallel version of Brandes' algorithm, executed on up to 96,000 CPU cores across multiple supercomputing systems. Bernaschi et al. [3] address the exact computation on multi-GPU

systems by proposing a two-dimensional graph decomposition strategy. Vella et al. [41] subsequently extend this line of research with the Dynamic Merging of Frontiers technique, which optimizes graph traversal by unifying exploration frontiers during BFS visits to avoid redundant computations, thereby improving performance on heterogeneous architectures. In parallel, within the context of large-scale distributed systems, Solomonik et al. [39] reformulate betweenness computation using linear algebra primitives, presenting an algorithm based on communication-efficient Sparse Matrix Multiplication. Hua et al. [21] are the first to propose a nearly optimal distributed algorithm for betweenness centrality under the CONGEST model, a standard message-passing abstraction where communication is restricted by limiting the size of messages sent across each edge to $O(\log n)$ bits per round. Hoang et al. [20] extend this line of work with the first nontrivial CONGEST algorithm for betweenness centrality on directed graphs, computing it together with all-pairs shortest paths in $O(n)$ rounds.

2.3 Graph Neural Networks

Graph neural networks have enabled the development of several learning-based methods for approximating centrality measures. Among the earliest GNN-based methods for betweenness-centrality approximation are those of Maurya et al. [30] and Fan et al. [12], which appeared concurrently. Maurya et al. [30] introduced GNN-Bet to approximate betweenness centrality, notably by pruning from the adjacency matrix those nodes that lie on no shortest path and by using graph convolutional layers while excluding the node's own feature vector during aggregation. This approach was later extended to closeness centrality in the journal version of the paper [31]. Fan et al. [12] proposed DrBC, a GNN combined with a gated recurrent unit (GRU). This work was further improved by Mirakyan [32], who proposed ABCDE, a deep graph convolutional network with a regularization scheme that progressively drops random edges in each convolutional block. Despite its depth, ABCDE uses fewer parameters than DrBC and improves performance in ranking nodes by betweenness centrality. Notably, ABCDE was not compared with Maurya et al. [30]. We provide a detailed architectural and experimental comparison with these models in Section 5.2. At a high level, BRAVA-GNN differs from them by replacing node-specific or single-hop degree features with multi-hop degree-mass features, maintaining a graph-size-independent parameterization, and supporting both directed and undirected graphs.

Beyond static graphs, recent work has also explored GNN-based centrality prediction in dynamic or temporal settings. Heeg and Scholtes [19] use De Bruijn graph neural networks to predict temporal centrality measures, including temporal betweenness.

3 Preliminaries

This section introduces the notation and background used throughout the paper: betweenness centrality, the node-ranking objective, multi-hop degree mass, and the hyperbolic random graph model used to construct part of our synthetic training distribution.

3.1 Betweenness Centrality

Given an unweighted graph $G = (V, E)$, either directed or undirected, the betweenness centrality $C_B(v)$ of a node $v \in V$ is defined

as:

$$C_B(v) = \sum_{\substack{s,t \in V \setminus \{v\} \\ s \neq t \\ \sigma_{st} > 0}} \frac{\sigma_{st}(v)}{\sigma_{st}}, \quad (1)$$

where σ_{st} is the number of shortest paths from node s to node t , and $\sigma_{st}(v)$ is the number of those paths that pass through node v . Pairs with no path from s to t are excluded from the sum and therefore contribute zero. In directed graphs, shortest paths are computed with respect to edge orientation; in undirected graphs, paths may traverse edges in either direction.

3.2 Problem Definition

Given an unweighted graph $G = (V, E)$, either directed or undirected, our goal is to learn a scoring function $f : V \rightarrow \mathbb{R}$ that assigns each node $v \in V$ a real-valued score $f(v)$. These scores induce a node ranking in descending order, where higher-ranked nodes are predicted to have higher betweenness centrality. The target ordering sorts nodes in decreasing $C_B(v)$, so that nodes with larger betweenness centrality appear earlier.

We formulate the task as a ranking problem rather than a regression problem: the objective is not to predict the absolute values of $C_B(v)$, but to preserve the relative ordering of nodes. Accordingly, model quality is evaluated by the Kendall τ_b correlation between the predicted ranking and the ground-truth betweenness ranking, which accounts for ties in centrality values. This formulation reflects many downstream uses of betweenness centrality, where identifying the most central nodes is more important than estimating their exact centrality scores.

3.3 Multi-hop Degree Mass

The m -th order degree mass of a node in a graph $G = (V, E)$ captures the cumulative connectivity around that node up to distance m . Originally introduced by Li et al. [28], it is defined as

$$\mathbf{d}^{(m)} = \left(\sum_{k=0}^m A^k \right) \mathbf{d}, \quad (2)$$

where A is the adjacency matrix and $\mathbf{d}$ is the degree vector. For directed graphs, degree masses are computed with respect to the matrix supplied to the feature extractor, using that matrix's row-degree vector. BRAVA-GNN applies this definition to both A and A^T to encode the two edge orientations.

For a node i , the i -th entry of $\mathbf{d}^{(m)}$ can be written as

$$d_i^{(m)} = \sum_{k=0}^m \sum_{j \in V} (A^k)_{ij} d_j. \quad (3)$$

Thus, in an unweighted graph, $(A^k)_{ij}$ counts the number of walks of length k from i to j , and $d_i^{(m)}$ aggregates the degrees of nodes reachable from i through walks of length at most m , weighted by the number of such walks. The resulting values are nonnegative integers for unweighted graphs. This quantity is closely related to multi-hop walk-based notions of node influence: nodes with large degree mass are connected, through short walks, to many high-degree nodes. Unlike random-walk-based embeddings or influence estimators, however, degree mass is a deterministic structural feature that can be computed by a small number of sparse

matrix-vector multiplications. Li et al. [28] empirically demonstrate a strong correlation between higher-order degree masses and betweenness centrality in real-world networks, motivating their use as input features for betweenness-ranking prediction.

3.4 Hyperbolic Random Graph Model

We use hyperbolic random graphs as part of our synthetic training distribution, because they reproduce structural properties commonly observed in real-world networks, including heterogeneous degree distributions, high clustering, and hierarchical organization. The model, introduced by Krioukov et al. [24], samples nodes in a hyperbolic disk, where radial coordinates are associated with node popularity or expected degree, and angular coordinates encode similarity. Edges are then formed preferentially between nodes that are close in the hyperbolic geometry.

In this model, the degree distribution follows a power law $P(k) \sim k^{-\gamma}$, with an exponent controlled by the geometry of the node distribution. In the zero-temperature formulation of Krioukov et al. [24], if the curvature is $K = -\zeta^2$ and α controls the radial density, then

$$\gamma = \frac{2\alpha}{\zeta} + 1. \quad (4)$$

Here, $\zeta > 0$ is the curvature scale and $\alpha > 0$ controls how strongly nodes concentrate near the boundary of the disk.

In our experiments, we use NetworKit's HyperbolicGenerator [40], which implements the subquadratic generator of von Looz and Meyerhenke [42]. This implementation exposes the model through four parameters: the number of nodes N , the average degree $\bar{k}$, the power-law exponent γ , and the temperature T , which controls clustering by smoothing the edge-formation rule. We therefore treat $\bar{k}$, γ , and T as the relevant generation parameters for constructing synthetic training graphs. As described in Section 5.1.1, $\bar{k}$ and γ are sampled from empirical real-world network statistics rather than chosen arbitrarily.

4 Proposed Method

BRAVA-GNN first simplifies the input graph with a shortest-path preprocessing heuristic, and then predicts betweenness rankings with a lightweight message-passing architecture based on multi-hop degree-mass features.

4.1 Preprocessing

We adopt the preprocessing heuristic introduced in Maurya et al. [30]. For undirected graphs, this rule removes nodes that are guaranteed not to lie on any shortest path. In particular, nodes satisfying either of the following two conditions are removed:

- (1) **Isolated or Leaf Nodes:** Any node with fewer than two neighbors has zero centrality, as betweenness is strictly defined for intermediate points.
- (2) **Clique Neighborhoods:** If a node's neighbors form a clique, any shortest path between them will favor the direct edge (length 1) over a path through the node (length 2).

For directed inputs, these two conditions are evaluated on the underlying undirected graph obtained by ignoring edge orientation. The retained graph is then the directed subgraph induced by the remaining vertices, so directed shortest paths, degree masses, and message

passing still use the directed adjacency. Thus, directionality is ignored only for the pruning heuristic, and the clique-neighborhood rule is not used as a directed shortest-path certificate. Observe that, while removed nodes have zero betweenness centrality in the undirected case, removing them can affect the betweenness values of the remaining nodes. In practice, this impact is negligible compared to the obtained simplification of the graph topology.

4.2 Architecture

BRAVA-GNN is a lightweight message-passing architecture designed to produce one scalar score per node. The model uses fixed-dimensional degree-mass features and two directional message-passing streams, allowing the same architecture to operate on graphs of arbitrary size and on both directed and undirected inputs. The complete procedure is summarized in Algorithm 2 and illustrated in Figure 1.

4.2.1 Input Features. As initial node features, we use the higher-order degree masses introduced in Section 3.3. For each node, we concatenate degree masses up to order five,

$$[\mathbf{d}^{(0)}, \mathbf{d}^{(1)}, \dots, \mathbf{d}^{(5)}],$$

and map the resulting six-dimensional vector to a hidden representation through a dense layer with ReLU activation. Since the feature dimension is fixed, the number of learnable parameters is independent of the number of nodes in the input graph.

4.2.2 Dual Message Passing. In directed graphs, betweenness centrality is determined by directed shortest paths. A node is central when it can connect sources that reach it to destinations reachable from it. To capture these two roles, BRAVA-GNN applies two parallel message-passing streams: an outgoing stream over A^T and an incoming stream over A . The two streams share weights, which keeps the architecture compact while applying the same aggregation rule to both directions.

Each stream consists of two message-passing layers. At each layer, a node aggregates hidden representations from its neighbors without adding its own hidden state, following the aggregation scheme used by GNN-Bet. A shared MLP maps the hidden representation at each layer to an intermediate scalar score.

4.2.3 Score Fusion. For each direction, intermediate scores from all layers are summed to obtain $y_{out}(v)$ and $y_{in}(v)$. The final score is computed as

$$y(v) = y_{in}(v) \cdot y_{out}(v).$$

This multiplicative fusion favors nodes that receive strong evidence from both incoming and outgoing neighborhoods, matching the intuition that high betweenness in directed graphs requires connecting many possible sources to many possible destinations. The fusion strategy is evaluated in the ablation study in Appendix 8.1.

5 Experimental Setup

This section describes the experimental protocol for evaluating BRAVA-GNN, including the training and test datasets, baselines, evaluation metric, loss function, implementation details, and hardware/software environment.

Algorithm 1 DegreeMassEmbedding.

```

1: Input: Adjacency matrix  $A$ 
2: Output: Initial embedding  $H$ .
3: Compute degree masses  $\mathbf{d}^{(0)}, \mathbf{d}^{(1)}, \mathbf{d}^{(2)}, \mathbf{d}^{(3)}, \mathbf{d}^{(4)}, \mathbf{d}^{(5)}$  from  $A$ 
   (Section 3.3)
4:  $H \leftarrow [\mathbf{d}^{(0)}, \mathbf{d}^{(1)}, \mathbf{d}^{(2)}, \mathbf{d}^{(3)}, \mathbf{d}^{(4)}, \mathbf{d}^{(5)}]$ 
5:  $H \leftarrow \text{ReLU}(H \cdot W_{6 \rightarrow 12}^{(0)})$ 
6: return  $H$ 

```

Algorithm 2 BRAVA-GNN.

```

1: Input: Adjacency Matrix  $A$ , Transpose Adjacency Matrix  $A^T$ ,
   number of message-passing layers  $L$ , Weight Matrix  $W^{(l)}$  at
   layer  $l$ .
2: Output: Betweenness score  $y$ .
3: Notation:  $\text{Norm}_2$  denotes row-wise  $L_2$  normalization.
4:  $H_{out} \leftarrow \text{DegreeMassEmbedding}(A^T)$ 
5:  $H_{in} \leftarrow \text{DegreeMassEmbedding}(A)$ 
6:  $H_{out}^{(0)} \leftarrow \text{Norm}_2(H_{out})$ 
7:  $H_{in}^{(0)} \leftarrow \text{Norm}_2(H_{in})$ 
8:  $y_{out} \leftarrow \text{MLP}(H_{out}^{(0)})$ 
9:  $y_{in} \leftarrow \text{MLP}(H_{in}^{(0)})$ 
10: for each layer  $l \in \{0, \dots, L-1\}$  do
11:    $H_{out}^{(l+1)} \leftarrow \text{Norm}_2(\text{ReLU}(A^T \cdot H_{out}^{(l)} \cdot W^{(l+1)}))$ 
12:    $H_{in}^{(l+1)} \leftarrow \text{Norm}_2(\text{ReLU}(A \cdot H_{in}^{(l)} \cdot W^{(l+1)}))$ 
13:    $y_{out} \leftarrow y_{out} + \text{MLP}(H_{out}^{(l+1)})$ 
14:    $y_{in} \leftarrow y_{in} + \text{MLP}(H_{in}^{(l+1)})$ 
15: end for
16: return  $y_{in} * y_{out}$ 

```

5.1 Datasets

5.1.1 Training set. To promote generalization across diverse network topologies, we train BRAVA-GNN on a hybrid synthetic dataset composed of directed and undirected scale-free graphs plus uniformly directed hyperbolic random graphs (UDHY).

We include directed scale-free graphs following the protocol of Maurya et al. [31], which has been widely used in previous GNN-based work on betweenness approximation. However, scale-free graphs alone capture only part of the structural variability observed in real networks. We therefore complement them with uniformly directed versions of hyperbolic random graphs, described in Section 3.4, which can reproduce several properties commonly found in real-world networks, including heterogeneous degree distributions, high clustering coefficients, and small-world structure [14, 17, 33].

Rather than selecting generation parameters arbitrarily, we adopt a data-driven parameterization strategy. We analyze a reference collection of 10 real-world networks [27, 38] to estimate empirical values of the average degree $\bar{k}$ and the power-law exponent γ ; these statistics are reported in Table 1. These empirical distributions are then used to parameterize the synthetic hyperbolic graphs.

Our final training set consists of 30 synthetic graphs, each with $N = 100,000$ nodes:

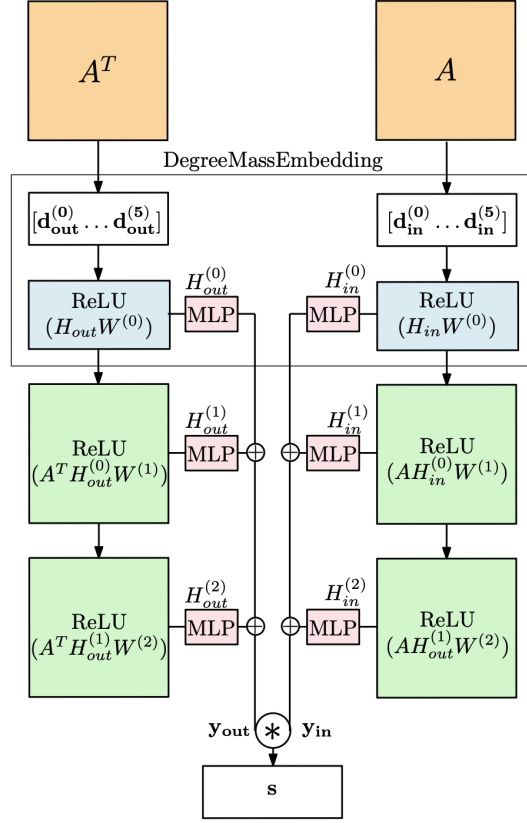


Figure 1: BRAVA-GNN Architecture Overview. The model utilizes degree masses up to the fifth order as input features, followed by two message-passing layers with ReLU activation and shared MLPs.

- **10 Directed Scale-Free Graphs:** generated using the configuration detailed in Maurya et al. [31].
- **10 Undirected Scale-Free Graphs:** generated using the same configuration and then symmetrized by replacing each directed edge $u \rightarrow v$ with the undirected pair $\{u, v\}$.
- **10 Directed Hyperbolic Random Graphs:** generated by sampling the temperature T uniformly from $(0, 0.5)$, while k and γ are drawn from the empirical distributions in Table 1. Each undirected edge $\{u, v\}$ is then oriented uniformly at random as either $u \rightarrow v$ or $v \rightarrow u$.

For each synthetic graph, exact betweenness centrality scores are computed and used to derive the ground-truth ranking labels.

5.1.2 Test set. For evaluation, we use 14 real-world networks from the Network Data Repository [38] and SNAP [27]. The benchmark spans eight domains: social, web, wiki, email, peer-to-peer, citation, authorship, and co-purchase networks. It includes both directed and undirected graphs and covers a broad range of graph sizes, from peer-to-peer networks with tens of thousands of nodes to large-scale graphs with millions of nodes and edges.

To ensure comparability with prior work, our test set includes the graphs used by Maurya et al. [31], as well as the subset used

by Mirakyan [32] and Fan et al. [12]: com-youtube, amazon, dblp, cit-Patents, and com-lj. Detailed node and edge counts for all test graphs are reported in Table 9 in the Appendix.

5.2 Baselines

We compare BRAVA-GNN with two families of baselines. The learning baselines are **GNN-Bet**, **DrBC**, and **ABCDE** [12, 30, 32]. The sampling baselines are **KADABRA**, **SILVAN**, and **Bavarian** [6, 11, 35]. ABCDE has previously been shown to outperform earlier sampling-based methods such as ABRA [37] and RK [36], as well as the learning-based Node2Vec baseline [16]; we therefore omit these earlier methods from direct comparison.

5.2.1 GNN-Bet. GNN-Bet [30] is the closest baseline to our work. We adopt its preprocessing heuristic and build on its idea of processing both edge directions. However, GNN-Bet uses learnable node-specific embeddings, making its number of parameters depend on the graph size and requiring padding to the largest graph in the training set. BRAVA-GNN instead uses fixed-dimensional degree-mass features, reducing the parameter count from $\approx 58M$ to $\approx 1.3k$ and allowing the same model to be applied to graphs of different sizes.

5.2.2 ABCDE. ABCDE [32] represents a significant step towards parameter-efficient centrality approximation. It uses a deep graph convolutional architecture regularized by *Progressive-DropEdge*, which progressively drops random edges in each convolutional block to reduce overfitting and over-smoothing. ABCDE uses node degree as its input feature and relies on multiple convolutional layers to aggregate wider neighborhood information. Compared with ABCDE, BRAVA-GNN uses higher-order degree-mass features directly and has a smaller parameter count ($\approx 1.3k$ vs. $\approx 72k$). ABCDE is designed for undirected graphs, whereas BRAVA-GNN is evaluated on both directed and undirected networks.

5.2.3 DrBC. DrBC [12] is a GNN-based model for betweenness-centrality ranking that uses degree-based initial node features. It aggregates information from neighboring nodes and updates node representations through a gated recurrent unit (GRU). To combine information from different propagation depths, DrBC applies max-pooling over the representations produced at each layer. Like ABCDE, DrBC is designed for undirected graphs.

5.2.4 KADABRA. KADABRA [6] is a randomized sampling algorithm for approximating betweenness centrality in directed and undirected graphs. It repeatedly samples shortest paths between pairs of nodes and updates the estimates of the nodes lying on the sampled paths. Its adaptive stopping criterion uses probabilistic confidence intervals to stop once the target approximation accuracy is reached. We use an error tolerance of $\epsilon = 0.01$ and a failure probability of $\delta = 0.1$, matching the configuration used for the other sampling-based baselines when applicable.

5.2.5 SILVAN. SILVAN [35] is a randomized shortest-path sampling algorithm for betweenness centrality approximation with probabilistic guarantees. Unlike KADABRA, it uses a progressive sampling scheme based on Monte Carlo Empirical Rademacher Averages and non-uniform bounds, refining the confidence estimates in a node-dependent way. This can reduce the number of

samples needed to obtain high-quality approximations. We set the approximation accuracy to $\epsilon = 0.01$ and the failure probability to $\delta = 0.05$, following the default confidence setting used in the original SILVAN implementation.

5.2.6 Bavarian. Bavarian [11] is a sampling-based algorithmic framework for betweenness centrality approximation. It combines shortest-path sampling with Monte Carlo Empirical Rademacher Averages and variance-aware probabilistic bounds to obtain tight, data-dependent approximation guarantees. We use the RK estimator variant (Riondato–Kornaropoulos [36]), with $\epsilon = 0.01$, $\delta = 0.1$, 100 Monte Carlo trials, and a progressive scaling factor of 2.0.

5.3 Loss Function

As motivated in Section 3.2, we formulate centrality prediction as a pairwise ranking problem rather than regression. We employ a *Margin Ranking Loss* to enforce the correct ordering of nodes based on their centrality scores. Given two nodes u and v , the loss is defined as:

$$\mathcal{L}(s_u, s_v, y) = \max(0, 1 - y \cdot (s_u - s_v)), \quad (5)$$

where s_u and s_v are the predicted scores generated by the model for nodes u and v , respectively. The label $y \in \{1, -1\}$ serves as the ranking indicator: we set $y = 1$ if u has strictly larger ground-truth betweenness than v , and $y = -1$ in the opposite case. Pairs with equal ground-truth betweenness are omitted from the loss because they do not define a strict preference. This formulation aligns with the pairwise hinge loss used in Learning to Rank frameworks [10].

5.4 Evaluation Metric

To evaluate the performance of our model in the node ranking task, we use Kendall’s τ_b coefficient. This metric measures the ordinal association between the predicted and the ground-truth rankings. The τ_b variant accounts for ties in the rankings, which is particularly relevant for betweenness centrality, where large subsets of nodes (e.g., peripheral nodes or symmetric structures) often share identical scores. Formally, the Kendall τ_b coefficient is defined as:

$$\tau_b = \frac{C - D}{\sqrt{(C + D + T)(C + D + U)}}, \quad (6)$$

where C denotes the number of *concordant pairs* (pairs of nodes (i, j) whose ranks agree in both the predicted and true rankings), D denotes the number of *discordant pairs* (pairs whose ranks are reversed), T is the number of ties only in the predicted ranking, and U is the number of ties only in the true ranking. The τ_b coefficient ranges from -1 (perfect disagreement) to 1 (perfect agreement), where a higher score indicates that the model has learned to preserve the topological hierarchy of the network. For readability, all reported τ_b values are scaled by 100 (i.e., -100 to 100).

5.5 Implementation Details

We train BRAVA-GNN for 10 epochs using the Adam optimizer with learning rate 5×10^{-3} . The final architecture uses $L = 2$ message-passing layers and hidden dimension $n_{hid} = 12$. The prediction MLP consists of three linear layers with dimensions $12 \rightarrow 24 \rightarrow 24 \rightarrow 1$, ReLU activations, and dropout $p = 0.3$. We apply row-wise L_2

Table 1: Reference networks for estimating γ , $\bar{k}$, and ρ (degree-betweenness correlation); dash = not computed.

Graph	#Nodes	#Edges	$\bar{k}$	γ	ρ
p2p-Gnutella30	36,682	88,328	4.8159	7.0192	0.9287
email-Enron	36,692	183,831	10.0202	2.3982	0.8157
musae-github	37,700	289,003	15.3317	2.5023	0.8884
soc-Slashdot0811	77,360	828,161	21.4106	2.1604	0.8442
gemsec-Facebook	50,515	819,090	32.4296	2.4494	0.7246
twitch-gamers	168,114	6,797,557	80.8684	2.1334	0.7553
com-DBLP	317,080	1,049,866	6.6221	3.5380	0.7200
web-NotreDame	325,729	1,469,679	9.0239	2.0757	0.2905
web-BerkStan	685,230	7,600,595	22.1841	2.6245	0.4503
com-Orkut	3,072,441	117,185,083	76.2814	2.2849	–

normalization after each message-passing activation to improve training stability.

The hyperparameters were selected through the ablation studies reported in Appendix 8.1, balancing ranking accuracy and parameter efficiency. The chosen configuration reaches $\tau_b = 85.5 \pm 0.5$, matching the best deeper settings while using substantially fewer parameters (Table 8). The case $L = 0$ corresponds to removing message passing entirely; in this setting, the model reduces to an MLP applied directly to the degree-mass features. The same ablations show that five-hop degree mass performs best among the tested feature orders (Table 6), and that dropout $p = 0.3$ gives the best performance among the tested rates 0.0–0.5 (Table 7).

To assess statistical robustness, we repeat training with three random seeds and report mean performance and standard deviation across independent runs. The complete training process takes approximately 18 minutes on a single NVIDIA Quadro RTX 8000 GPU.

5.6 Hardware and Software

BRAVA-GNN is implemented in PyTorch [34]. We use NetworkX [18] for graph generation and data processing, and NetworKit [40] to compute exact betweenness centrality values.

All runs reported in this paper, including training, inference, and wall-clock baseline evaluations, were executed on the same hardware platform: 2× Intel Xeon Gold 6230R CPUs (52 cores total), 384 GiB of RAM, and 4× NVIDIA Quadro RTX 8000 GPUs with 45 GiB of usable GPU memory each. Each run used a single GPU, and the machine was reserved exclusively during execution to make wall-clock timings comparable across methods. Reported inference times measure a single forward pass. For BRAVA-GNN, they also include the computation of degree-mass features $[d^{(0)}, \dots, d^{(5)}]$ from the adjacency matrix.

6 Results

Tables 2 and 3 compare BRAVA-GNN against the baselines in ranking accuracy (Kendall τ_b) and inference time (seconds), respectively. Parameter count is reported separately in Table 4. BRAVA-GNN attains the best Kendall τ_b on all 14 test graphs (Table 2), improving upon the strongest baseline by 9.7% on average. The gain is consistent across both regimes, with +16.1% on undirected graphs

Table 2: Ranking accuracy (Kendall τ_b , $\uparrow$; mean $\pm$ std. over 3 seeds). Best/second-best are bolded/underlined. % Imp. is BRAVA-GNN’s gain over the strongest baseline. Dash = unavailable; ABCDE/DrBC are undirected-only; Bavarian fails due to out-of-memory errors.

	GNN-Bet	ABCDE	DrBC	KADABRA	SILVAN	Bavarian	BRAVA-GNN	% Imp.
Undirected								
p2p-Gnutella31	<u>89.3 $\pm$ 1.1</u>	72.6 $\pm$ 0.8	78.2 $\pm$ 0.0	72.0 $\pm$ 0.0	66.9 $\pm$ 1.1	68.8 $\pm$ 0.1	89.7 $\pm$ 0.7	+0.4%
com-youtube	<u>76.3 $\pm$ 0.2</u>	56.2 $\pm$ 0.3	49.5 $\pm$ 0.0	32.3 $\pm$ 0.1	44.7 $\pm$ 0.0	49.0 $\pm$ 5.5	92.2 $\pm$ 0.1	+20.9%
amazon	<u>68.9 $\pm$ 0.2</u>	65.9 $\pm$ 1.2	66.2 $\pm$ 0.0	19.0 $\pm$ 0.1	24.7 $\pm$ 0.3	20.4 $\pm$ 0.1	85.9 $\pm$ 1.7	+24.6%
cit-Patents	57.5 $\pm$ 0.2	66.7 $\pm$ 1.0	<u>68.2 $\pm$ 0.0</u>	18.3 $\pm$ 0.0	17.9 $\pm$ 0.2	15.4 $\pm$ 0.0	74.0 $\pm$ 0.2	+8.6%
com-lj	<u>68.8 $\pm$ 0.4</u>	65.3 $\pm$ 1.2	62.2 $\pm$ 0.0	16.7 $\pm$ 0.0	28.9 $\pm$ 0.0	–	80.5 $\pm$ 0.2	+17.0%
dblp	76.2 $\pm$ 0.7	<u>78.6 $\pm$ 0.6</u>	69.1 $\pm$ 0.0	18.1 $\pm$ 0.0	17.2 $\pm$ 1.1	–	85.0 $\pm$ 1.3	+8.2%
Undirected AVG	<u>72.8</u>	67.6	65.6	29.4	33.4	–	84.6	+16.1%
Directed								
soc-Epinions1	<u>87.9 $\pm$ 0.2</u>	–	–	53.7 $\pm$ 0.0	49.0 $\pm$ 0.5	56.6 $\pm$ 0.2	92.6 $\pm$ 0.1	+5.3%
soc-Slashdot0902	<u>81.4 $\pm$ 0.6</u>	–	–	51.9 $\pm$ 0.0	55.8 $\pm$ 0.1	61.6 $\pm$ 0.0	90.3 $\pm$ 0.4	+10.9%
email-EuAll	<u>98.9 $\pm$ 0.1</u>	–	–	31.7 $\pm$ 0.2	32.2 $\pm$ 1.1	30.0 $\pm$ 0.2	99.2 $\pm$ 0.0	+0.3%
web-Google	<u>77.6 $\pm$ 0.2</u>	–	–	34.0 $\pm$ 0.0	30.6 $\pm$ 0.1	39.2 $\pm$ 0.2	79.8 $\pm$ 0.4	+2.7%
soc-Pokec	<u>76.0 $\pm$ 0.3</u>	–	–	22.2 $\pm$ 0.3	35.8 $\pm$ 0.2	–	83.3 $\pm$ 0.6	+9.6%
wiki-topcats	<u>58.4 $\pm$ 1.5</u>	–	–	21.5 $\pm$ 0.0	28.4 $\pm$ 0.4	24.4 $\pm$ 0.1	63.1 $\pm$ 0.9	+8.1%
wiki-Talk	<u>98.8 $\pm$ 0.1</u>	–	–	12.1 $\pm$ 0.0	5.8 $\pm$ 0.2	8.4 $\pm$ 0.3	99.3 $\pm$ 0.0	+0.5%
soc-LiveJournal1	<u>75.9 $\pm$ 0.3</u>	–	–	14.7 $\pm$ 0.0	20.5 $\pm$ 0.8	–	82.8 $\pm$ 0.2	+9.0%
Directed AVG	<u>81.9</u>	–	–	30.2	32.3	–	86.3	+5.4%
AVG	<u>78.0</u>	–	–	29.9	32.8	–	85.5	+9.7%

Table 3: Inference time (seconds, $\downarrow$; mean $\pm$ std. over 3 seeds). Brandes is exact (single run). Best/second-best are bolded/underlined. Entries shown as < 0.05 round to 0.0s at one-decimal precision; speedups use unrounded measurements. AVG speedup is the median per-graph best-baseline-to-BRAVA ratio. Dash = unavailable; ABCDE/DrBC are undirected-only; Bavarian ran out of memory; Brandes timed out after 24h.

	GNN-Bet	ABCDE	DrBC	KADABRA	SILVAN	Bavarian	Brandes	BRAVA-GNN	Speedup $\uparrow$
Undirected									
p2p-Gnutella31	0.6 $\pm$ 0.0	0.4 $\pm$ 0.0	<u>0.2 $\pm$ 0.0</u>	0.6 $\pm$ 0.1	0.6 $\pm$ 0.1	13.7 $\pm$ 0.9	48.3	< 0.05	13.8x
com-youtube	<u>0.6 $\pm$ 0.0</u>	10.7 $\pm$ 0.3	4.5 $\pm$ 0.3	15.8 $\pm$ 0.6	17.1 $\pm$ 2.5	1312.6 $\pm$ 442.4	3416.7	0.3 $\pm$ 0.0	2.1x
amazon	0.7 $\pm$ 0.0	24.0 $\pm$ 1.5	11.2 $\pm$ 0.4	28.4 $\pm$ 1.3	35.2 $\pm$ 2.4	990.0 $\pm$ 10.6	14061.1	<u>0.7 $\pm$ 0.1</u>	1.0x
cit-Patents	0.9 $\pm$ 0.0	46.4 $\pm$ 3.1	16.9 $\pm$ 1.1	53.9 $\pm$ 0.2	56.5 $\pm$ 0.9	1216.4 $\pm$ 4.9	23003.0	<u>2.1 $\pm$ 0.5</u>	0.4x
com-lj	1.4 $\pm$ 0.0	88.4 $\pm$ 2.7	26.4 $\pm$ 0.5	88.9 $\pm$ 1.5	91.3 $\pm$ 0.5	–	143715.3	<u>3.8 $\pm$ 0.7</u>	0.4x
dblp	0.7 $\pm$ 0.0	28.0 $\pm$ 0.1	11.9 $\pm$ 0.2	59.9 $\pm$ 1.4	58.6 $\pm$ 5.5	–	15102.6	<u>1.0 $\pm$ 0.2</u>	0.6x
Undirected AVG	0.8	33.0	11.9	41.2	43.2	–	33224.5	<u>1.3</u>	0.8x
Directed									
soc-Epinions1	<u>0.6 $\pm$ 0.0</u>	–	–	1.3 $\pm$ 0.2	1.0 $\pm$ 0.0	59.1 $\pm$ 1.7	33.6	< 0.05	24.5x
soc-Slashdot0902	<u>0.7 $\pm$ 0.0</u>	–	–	1.7 $\pm$ 0.0	1.8 $\pm$ 0.3	57.5 $\pm$ 23.8	97.7	< 0.05	15.9x
email-EuAll	<u>0.6 $\pm$ 0.0</u>	–	–	2.4 $\pm$ 0.6	2.1 $\pm$ 0.5	198.7 $\pm$ 1.9	96.4	< 0.05	23.4x
web-Google	<u>0.7 $\pm$ 0.0</u>	–	–	17.3 $\pm$ 1.3	16.9 $\pm$ 0.4	6322.9 $\pm$ 1032.3	4191.0	0.2 $\pm$ 0.0	2.9x
soc-Pokec	1.4 $\pm$ 0.0	–	–	82.5 $\pm$ 1.3	83.0 $\pm$ 0.1	–	80363.8	<u>1.7 $\pm$ 0.4</u>	0.8x
wiki-topcats	1.3 $\pm$ 0.0	–	–	61.6 $\pm$ 2.1	61.4 $\pm$ 1.4	2083.2 $\pm$ 702.9	69991.3	<u>1.5 $\pm$ 0.2</u>	0.9x
wiki-Talk	<u>0.6 $\pm$ 0.0</u>	–	–	30.1 $\pm$ 1.8	30.6 $\pm$ 0.7	1207.3 $\pm$ 209.8	9909.6	0.3 $\pm$ 0.1	2.0x
soc-LiveJournal1	2.3 $\pm$ 0.0	–	–	161.3 $\pm$ 4.4	163.6 $\pm$ 2.8	–	–	<u>3.8 $\pm$ 0.7</u>	0.6x
Directed AVG	<u>1.0</u>	–	–	44.8	45.0	–	–	1.0	2.5x
AVG	0.9	–	–	43.3	44.3	–	–	<u>1.1</u>	1.5x

and +5.4% on directed graphs, and peak per-graph improvements of +24.6% and +10.9% respectively. ABCDE and DrBC cannot process directed adjacencies and are therefore absent from the directed columns, underscoring BRAVA-GNN’s broader applicability.

Parameter efficiency. Table 4 reports the number of learnable parameters for each model. With only 1,297 parameters, BRAVA-GNN is 56 $\times$ smaller than ABCDE, 96 $\times$ smaller than DrBC, and nearly five orders of magnitude smaller than GNN-Bet, whose embedding-based design ties model size to the largest training graph. This

compactness does not come at the cost of runtime: as shown in Table 3, BRAVA-GNN’s inference latency is competitive with the fastest GNN baseline on every dataset and remains well below the sampling-based methods on the largest graphs.

Table 4: Number of learnable parameters. GNN-Bet [30] scales with graph size ($|V|$), while DrBC [12], ABCDE [32], and our model are size-invariant. Notably, BRAVA-GNN requires $\sim 56\times$ fewer parameters than ABCDE.

Model	# Parameters
GNN-Bet	58,172,425
DrBC	124,225
ABCDE	72,477
BRAVA-GNN	1,297

7 Conclusion

In this work, we introduced BRAVA-GNN, a lightweight framework for approximating betweenness centrality rankings. The method is based on two key design choices. First, it replaces the node-specific embeddings used by previous GNN-based approaches, whose parameterization may depend on the input graph size, with size-invariant multi-hop degree-mass features. Second, it is also trained on uniformly directed hyperbolic random graphs, whose power-law degree distributions and clustering patterns better reflect the structure of real-world networks than the scale-free synthetic graphs commonly used in prior work.

These choices enable BRAVA-GNN to achieve strong generalization with only about 1,300 trainable parameters, substantially fewer than existing GNN-based baselines. Across all 14 test graphs, covering both directed and undirected settings, BRAVA-GNN consistently outperforms larger models in betweenness ranking accuracy.

Future work will investigate whether the same topological learning principle can be extended to other path-based centrality measures, such as closeness centrality. Another promising direction is the adaptation of BRAVA-GNN to temporal graphs, where centrality rankings evolve over time and efficient updates become essential.

8 Appendix

8.1 Ablation Study

To assess the impact of the main design choices of BRAVA-GNN, we perform ablation studies on the initial node features, the synthetic training distribution, the number of message-passing layers, the hidden dimension, the fusion mechanism, and the dropout rate. All results are averaged over three independent runs. The final configuration is selected by balancing accuracy and parameter efficiency; selected hyperparameters are marked with a $\dagger$.

Training distribution. Since our goal is to train a model that generalizes to both directed and undirected graphs, we construct the training set using synthetic graphs from both regimes. We start from directed Bollobás scale-free graphs (**BO**) [4], following [30], and consider several additional graph families: symmetrized Bollobás scale-free graphs (**SBO**) described in Section 5.1.1, Barabási–Albert

graphs (**BA**) [2], hyperbolic random graphs (**HY**) as described in Section 3.4, and directed variants of the hyperbolic model. For the latter, we test both a uniformly directed version, where each undirected hyperbolic edge is assigned a random orientation (**UDHY**), and the directed m -nearest-neighbor (**mNN**) construction proposed by [23]. In the m NN variant, each node creates outgoing edges toward its m closest nodes in the hyperbolic space. We sample m uniformly from $\{12, 16, 20\}$, since smaller values often produce disconnected graphs.

We evaluate multiple training mixtures, reported in Table 5. We first train **BO**, **SBO**, **HY**, **UDHY** independently. We then group the latter training sets according to edge orientation, obtaining a directed-only mix (**Dir-only: BO, UDHY**), an undirected-only mix (**Undir-only: SBO, HY**), and a combined mix containing all four models (**All-4**). Starting from **All-4**, we also evaluate leave-one-out variants, each obtained by removing one graph model; for example, **-HY** denotes the mixture containing all families except **HY**, and corresponds to the final configuration selected for BRAVA-GNN ($\dagger$). Finally, we assess alternative generation choices: **mNN-swap** replaces **UDHY** with the m NN hyperbolic variant, **BO+SBO+mNN** combines Bollobás scale-free graphs with the m NN variant, and **BA-swap** replaces **SBO** with graphs generated by the Barabási–Albert preferential attachment model.

Feature initialization. Table 6 shows that degree-mass features are crucial for BRAVA-GNN. Using only the degree is already competitive, but adding higher-order degree masses improves ranking accuracy. The best performance is obtained by using degree masses up to order five, while using higher orders gives no further benefit. As a control, we also consider *random-6*, where each node v is initialized as

$$\mathbf{x}_v^{\text{rand}} = [u_{v,1}, \dots, u_{v,6}], \quad u_{v,i} \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}(0, 1).$$

The lower performance of this baseline confirms that the gain comes from the structural information encoded by degree masses, not only from the GNN architecture.

Architecture. The remaining ablations show that a small architecture is sufficient. Increasing the number of layers or the hidden dimension provides only marginal gains compared to the increase in model size. We therefore select $L = 2$ and $n_{\text{hid}} = 12$, which offers the best trade-off between accuracy and compactness. Shared incoming and outgoing encoders further reduce the number of parameters with negligible loss, while multiplicative fusion performs better than concatenation, supporting its use to combine incoming and outgoing flow information.

Overall, the ablation results support the three main choices of BRAVA-GNN: multi-hop degree-mass features, mixed synthetic training graphs, and a compact directional architecture.

8.2 Additional information on datasets

Table 9 lists the number of nodes and edges for all the graphs in the test set.

Table 5: Ablation: Training Distribution (accuracy).

Graph	BO	SBO	UDHY	HY	-SBO	-BO	-UDHY	-HY [†]	Dir-only	Undir-only	BO+HY	All-4	mNN-swap	BO+SBO+mNN	BA-swap
Undirected															
p2p-Gnutella31	90.7 ± 2.0	91.4 ± 2.4	89.4 ± 0.3	88.8 ± 0.6	91.1 ± 0.3	88.9 ± 3.3	88.4 ± 4.0	<u>93.2 ± 1.2</u> [†]	91.8 ± 0.9	89.7 ± 1.9	87.4 ± 0.1	89.5 ± 1.4	89.7 ± 1.0	91.6 ± 0.7	94.5 ± 1.1
com-youtube	81.9 ± 1.2	91.8 ± 0.8	89.2 ± 0.3	88.5 ± 0.8	90.9 ± 0.1	89.2 ± 0.8	91.2 ± 0.2	91.0 ± 0.5 [†]	89.3 ± 0.9	<u>91.3 ± 0.8</u>	87.8 ± 0.9	90.8 ± 0.1	90.2 ± 0.1	90.2 ± 0.7	87.8 ± 0.4
amazon	75.1 ± 1.6	86.6 ± 1.5	76.7 ± 0.2	75.7 ± 1.8	82.3 ± 0.8	77.7 ± 0.8	83.9 ± 1.1	82.8 ± 2.1 [†]	78.4 ± 0.8	<u>85.1 ± 1.2</u>	77.1 ± 2.2	82.0 ± 1.1	81.0 ± 1.6	85.1 ± 0.8	80.8 ± 1.6
cit-Patents	71.0 ± 1.1	71.1 ± 4.0	72.1 ± 0.7	71.9 ± 1.5	<u>73.8 ± 0.7</u>	72.6 ± 1.5	72.1 ± 2.0	72.8 ± 0.7 [†]	74.1 ± 1.1	72.3 ± 0.8	70.8 ± 1.0	73.4 ± 0.5	73.1 ± 0.2	70.2 ± 2.0	72.2 ± 0.5
com-lj	73.3 ± 0.6	75.4 ± 3.4	79.8 ± 0.6	77.7 ± 0.6	<u>80.2 ± 0.2</u>	79.4 ± 0.3	79.8 ± 1.0	79.5 ± 0.2 [†]	79.8 ± 0.4	80.4 ± 0.6	78.8 ± 0.7	79.8 ± 0.7	79.7 ± 0.0	77.5 ± 0.4	75.6 ± 1.1
dblp	83.9 ± 3.8	<u>86.4 ± 1.1</u>	83.5 ± 1.8	83.0 ± 1.6	82.8 ± 1.4	83.4 ± 1.7	82.9 ± 1.7	85.6 ± 1.7 [†]	86.3 ± 1.7	82.1 ± 0.7	79.8 ± 2.3	84.3 ± 0.2	80.1 ± 1.3	82.2 ± 1.4	87.1 ± 1.4
Directed															
soc-Epinions1	88.4 ± 0.2	91.6 ± 0.4	92.1 ± 0.3	90.9 ± 0.4	<u>92.4 ± 0.2</u>	91.8 ± 0.3	92.0 ± 0.3	92.7 ± 0.1 [†]	92.1 ± 0.3	92.1 ± 0.3	91.1 ± 0.3	92.2 ± 0.2	92.1 ± 0.1	92.1 ± 0.2	91.1 ± 0.3
soc-Slashdot0902	84.2 ± 0.5	<u>89.2 ± 0.3</u>	88.4 ± 0.1	86.9 ± 0.7	88.4 ± 0.5	87.2 ± 0.4	88.3 ± 0.3	89.6 ± 0.5 [†]	88.7 ± 0.5	88.4 ± 0.1	85.2 ± 0.9	88.5 ± 0.3	87.6 ± 0.9	88.5 ± 0.6	87.6 ± 0.3
email-EuAll	99.0 ± 0.1	99.1 ± 0.0	99.1 ± 0.0	99.0 ± 0.1	99.1 ± 0.0	99.1 ± 0.0	99.0 ± 0.1	99.1 ± 0.0 [†]	<u>99.1 ± 0.0</u>	99.0 ± 0.0	99.0 ± 0.0	99.1 ± 0.0	99.0 ± 0.0	99.1 ± 0.0	99.1 ± 0.0
web-Google	77.6 ± 0.1	78.4 ± 0.5	79.3 ± 0.7	78.6 ± 0.6	<u>79.5 ± 0.2</u>	80.1 ± 0.4	79.3 ± 0.6	79.2 ± 0.3 [†]	79.3 ± 0.2	79.2 ± 0.2	79.4 ± 0.7	79.4 ± 0.1	79.1 ± 0.1	78.7 ± 0.2	77.6 ± 0.3
soc-Pokec	78.0 ± 0.7	76.2 ± 4.4	82.4 ± 0.4	81.5 ± 0.4	82.6 ± 0.1	82.3 ± 0.1	82.2 ± 0.6	82.3 ± 0.8 [†]	82.4 ± 0.4	82.4 ± 0.2	81.3 ± 0.5	<u>82.5 ± 0.2</u>	81.8 ± 0.5	77.4 ± 0.9	80.6 ± 1.0
wiki-topcats	62.2 ± 1.1	58.5 ± 4.4	<u>61.6 ± 1.7</u>	61.1 ± 1.2	61.1 ± 0.8	61.5 ± 1.4	61.3 ± 0.8	60.4 ± 1.6 [†]	61.2 ± 1.3	60.4 ± 0.1	58.4 ± 1.4	61.5 ± 0.8	60.5 ± 2.9	54.4 ± 1.3	59.4 ± 1.0
wiki-Talk	99.0 ± 0.1	99.3 ± 0.0	99.3 ± 0.0	99.1 ± 0.0	99.3 ± 0.0	99.3 ± 0.0	99.2 ± 0.0	99.3 ± 0.0 [†]	99.3 ± 0.0	99.2 ± 0.1	99.2 ± 0.0	99.3 ± 0.0	99.2 ± 0.0	99.3 ± 0.0	99.2 ± 0.0
soc-LiveJournal1	77.5 ± 0.5	80.1 ± 1.0	82.2 ± 0.5	80.7 ± 0.3	<u>82.6 ± 0.0</u>	82.1 ± 0.3	82.4 ± 0.7	82.3 ± 0.1 [†]	82.3 ± 0.2	82.7 ± 0.4	81.4 ± 0.4	82.4 ± 0.5	82.1 ± 0.1	79.8 ± 0.7	79.8 ± 0.4
AVG	81.6 ± 1.0	83.9 ± 1.7	84.0 ± 0.5	83.1 ± 0.8	<u>84.7 ± 0.4</u>	83.9 ± 0.8	84.4 ± 1.0	85.0 ± 0.7 [†]	84.6 ± 0.6	84.6 ± 0.5	82.6 ± 0.8	84.6 ± 0.4	84.0 ± 0.6	83.3 ± 0.7	83.8 ± 0.7

Table 6: Ablation: Feature Initialization (accuracy). “random-6” denotes a six-dimensional random feature vector $\mathbf{x} \sim \mathcal{U}([0, 1]^6)$.

Graph	$[d^{(0)}]$	$[d^{(0)} d^{(1)}]$	$[\dots d^{(2)}]$	$[\dots d^{(3)}]$	$[\dots d^{(4)}]$	$[\dots d^{(5)}]$	$[\dots d^{(11)}]$	$[d^{(0)}, Ad^{(0)}, \dots, A^5 d^{(0)}]$	random-6
Undirected									
p2p-Gnutella31	89.5 ± 1.8	<u>90.2 ± 0.3</u>	93.2 ± 1.2	90.1 ± 1.0	90.0 ± 1.0	89.7 ± 0.7	88.0 ± 3.5	89.6 ± 2.0	80.0 ± 0.3
com-youtube	91.3 ± 0.1	91.0 ± 0.4	91.0 ± 0.5	91.8 ± 0.3	91.8 ± 0.2	92.2 ± 0.1	91.4 ± 0.4	<u>92.2 ± 0.1</u>	73.2 ± 0.2
amazon	82.2 ± 0.6	82.2 ± 1.5	82.8 ± 2.1	82.9 ± 1.9	86.6 ± 0.4	<u>85.9 ± 1.7</u>	83.1 ± 2.7	85.6 ± 1.0	75.0 ± 0.1
cit-Patents	73.0 ± 2.2	73.2 ± 1.7	72.8 ± 0.7	72.1 ± 2.6	71.2 ± 4.1	74.0 ± 0.2	<u>73.3 ± 1.8</u>	73.0 ± 1.1	72.5 ± 0.1
com-lj	79.6 ± 0.9	80.1 ± 0.7	79.5 ± 0.2	79.7 ± 0.8	80.0 ± 0.8	80.5 ± 0.2	79.9 ± 0.5	<u>80.3 ± 0.2</u>	74.9 ± 0.2
dblp	84.6 ± 1.2	85.5 ± 1.8	<u>85.6 ± 1.7</u>	84.3 ± 2.2	85.6 ± 1.2	85.0 ± 1.3	84.6 ± 1.5	83.9 ± 1.7	77.7 ± 0.2
Directed									
soc-Epinions1	92.6 ± 0.0	92.5 ± 0.1	<u>92.7 ± 0.1</u>	92.7 ± 0.1	92.5 ± 0.2	92.6 ± 0.1	92.3 ± 0.2	92.6 ± 0.1	70.8 ± 0.1
soc-Slashdot0902	89.6 ± 0.5	89.7 ± 0.5	89.6 ± 0.5	89.8 ± 0.2	90.1 ± 0.3	90.3 ± 0.4	89.7 ± 0.3	<u>90.2 ± 0.6</u>	80.4 ± 0.1
email-EuAll	99.0 ± 0.0	99.1 ± 0.1	99.1 ± 0.0	99.1 ± 0.0	<u>99.1 ± 0.0</u>	99.2 ± 0.0	99.1 ± 0.0	99.1 ± 0.0	27.4 ± 0.0
web-Google	78.8 ± 0.1	78.8 ± 0.3	79.2 ± 0.3	<u>79.8 ± 0.4</u>	79.5 ± 0.5	79.8 ± 0.4	79.4 ± 0.3	80.0 ± 0.4	64.6 ± 0.1
soc-Pokec	81.4 ± 1.3	82.5 ± 0.5	82.3 ± 0.8	81.8 ± 0.8	82.7 ± 0.3	83.3 ± 0.6	82.4 ± 1.3	<u>83.1 ± 0.4</u>	78.7 ± 0.2
wiki-topcats	62.3 ± 0.5	61.9 ± 1.5	60.4 ± 1.6	60.0 ± 2.4	62.5 ± 1.2	63.1 ± 0.9	<u>63.4 ± 0.7</u>	62.5 ± 0.9	63.5 ± 0.1
wiki-Talk	99.3 ± 0.0	99.3 ± 0.0	99.3 ± 0.0	99.3 ± 0.0	99.3 ± 0.0	99.3 ± 0.0	99.3 ± 0.0	<u>99.3 ± 0.0</u>	28.2 ± 0.0
soc-LiveJournal1	82.3 ± 0.5	82.3 ± 0.5	82.3 ± 0.1	82.2 ± 0.6	82.3 ± 0.4	82.8 ± 0.2	82.4 ± 0.3	<u>82.6 ± 0.2</u>	75.8 ± 0.2
AVG	84.7 ± 0.7	84.9 ± 0.7	85.0 ± 0.7	84.7 ± 1.0	85.2 ± 0.7	85.5 ± 0.5	84.9 ± 1.0	<u>85.3 ± 0.6</u>	67.3 ± 0.1

Table 7: Ablation: dropout, encoder sharing, and fusion strategy (accuracy).

Graph	Dropout						Encoder		Fusion	
	0.0	0.1	0.2	0.3 [†]	0.4	0.5	Shared [†]	Unshared	Add	Cat-MLP
Undirected										
p2p-Gnutella31	88.9 ± 0.9	89.0 ± 0.8	88.7 ± 0.5	89.7 ± 0.7	91.5 ± 0.1	<u>90.4 ± 1.0</u>	89.7 ± 0.7	91.5 ± 0.4	<u>90.1 ± 1.5</u>	89.6 ± 0.4
com-youtube	91.5 ± 0.3	92.3 ± 0.4	92.1 ± 0.3	<u>92.2 ± 0.1</u>	91.7 ± 0.1	91.3 ± 0.4	92.2 ± 0.1	91.7 ± 0.1	<u>91.9 ± 0.4</u>	91.7 ± 0.7
amazon	83.2 ± 1.1	<u>85.6 ± 0.6</u>	84.0 ± 1.1	85.9 ± 1.7	84.5 ± 0.5	83.9 ± 2.9	85.9 ± 1.7	84.8 ± 1.0	<u>85.2 ± 2.3</u>	83.0 ± 3.1
cit-Patents	72.8 ± 0.8	<u>73.7 ± 2.3</u>	72.2 ± 0.6	74.0 ± 0.2	72.9 ± 1.5	72.2 ± 2.1	<u>74.0 ± 0.2</u>	74.1 ± 1.4	73.4 ± 0.2	71.9 ± 1.2
com-lj	79.5 ± 0.4	<u>80.3 ± 1.4</u>	80.1 ± 1.1	80.5 ± 0.2	79.5 ± 0.6	79.5 ± 0.3	80.5 ± 0.2	<u>80.5 ± 0.5</u>	80.5 ± 1.0	78.0 ± 2.1
dblp	80.3 ± 1.8	82.8 ± 2.0	83.0 ± 1.4	85.0 ± 1.3	<u>86.0 ± 0.5</u>	86.2 ± 2.8	85.0 ± 1.3	86.0 ± 0.2	<u>85.4 ± 0.9</u>	84.8 ± 1.2
Directed										
soc-Epinions1	92.3 ± 0.2	92.4 ± 0.2	92.4 ± 0.2	92.6 ± 0.1	92.4 ± 0.1	<u>92.5 ± 0.1</u>	<u>92.6 ± 0.1</u>	92.6 ± 0.2	92.7 ± 0.3	92.4 ± 0.5
soc-Slashdot0902	89.9 ± 0.2	90.0 ± 0.2	90.0 ± 0.5	90.3 ± 0.4	<u>90.2 ± 0.2</u>	89.8 ± 0.5	90.3 ± 0.4	89.5 ± 0.3	89.8 ± 0.6	<u>89.8 ± 0.1</u>
email-EuAll	99.0 ± 0.1	<u>99.1 ± 0.0</u>	<u>99.1 ± 0.0</u>	99.2 ± 0.0	99.1 ± 0.0	99.1 ± 0.0	<u>99.2 ± 0.0</u>	99.1 ± 0.0	99.1 ± 0.0	99.2 ± 0.0
web-Google	80.8 ± 0.3	<u>80.5 ± 0.2</u>	80.3 ± 0.1	79.8 ± 0.4	79.5 ± 0.1	79.1 ± 0.2	79.8 ± 0.4	79.3 ± 0.2	79.2 ± 0.3	<u>79.8 ± 0.2</u>
soc-Pokec	83.1 ± 0.3	83.6 ± 0.8	82.9 ± 0.1	<u>83.3 ± 0.6</u>	82.9 ± 0.4	82.3 ± 0.5	83.3 ± 0.6	82.7 ± 0.6	<u>82.9 ± 0.2</u>	82.6 ± 0.3
wiki-topcats	62.0 ± 0.9	<u>63.0 ± 1.9</u>	62.8 ± 2.0	63.1 ± 0.9	63.0 ± 1.0	61.9 ± 1.1	63.1 ± 0.9	64.1 ± 0.4	<u>63.2 ± 1.2</u>	61.6 ± 1.5
wiki-Talk	99.2 ± 0.0	99.3 ± 0.0	<u>99.3 ± 0.0</u>	99.3 ± 0.0	99.3 ± 0.0	99.3 ± 0.0	99.3 ± 0.0	99.3 ± 0.0	99.3 ± 0.0	<u>99.3 ± 0.0</u>
soc-LiveJournal1	82.4 ± 0.2	82.9 ± 0.6	82.7 ± 0.6	<u>82.8 ± 0.2</u>	82.2 ± 0.1	82.1 ± 0.1	<u>82.8 ± 0.2</u>	82.6 ± 0.2	82.9 ± 0.4	81.5 ± 1.3
AVG	84.6 ± 0.5	85.3 ± 0.8	85.0 ± 0.6	85.5 ± 0.5	<u>85.3 ± 0.4</u>	85.0 ± 0.9	<u>85.5 ± 0.5</u>	85.6 ± 0.4	85.4 ± 0.7	84.6 ± 0.9

Table 8: Ablation: Depth × Width (accuracy).

$L \setminus n_{hid}$	8	12	16	32
0	81.5 ± 1.0	81.8 ± 0.5	81.7 ± 0.8	81.8 ± 0.6
1	84.9 ± 0.7	84.1 ± 1.9	85.0 ± 0.9	85.1 ± 0.9
2	85.5 ± 0.5	<u>85.5 ± 0.5</u>	85.2 ± 0.9	85.4 ± 0.8
4	84.7 ± 0.6	84.8 ± 0.8	85.5 ± 0.6	85.5 ± 0.8
6	84.6 ± 0.6	84.8 ± 0.6	84.7 ± 0.9	85.4 ± 0.9

Table 9: Node and edge counts for the 14 real-world test graphs.

Graph	#Nodes	#Edges
Undirected		
p2p-Gnutella31	62,586	147,892
com-youtube	1,134,890	2,987,624
amazon	2,146,057	5,743,146
cit-Patents	3,764,117	16,511,741
com-lj	3,997,962	34,681,189
dblp	4,000,148	8,649,011
Directed		
soc-Epinions1	75,879	508,837
soc-Slashdot0902	82,168	948,464
email-EuAll	265,214	420,045
web-Google	875,713	5,105,039
soc-Pokec	1,632,803	30,622,564
wiki-topcats	1,791,489	28,511,807
wiki-Talk	2,394,385	5,021,410
soc-LiveJournal1	4,847,571	68,993,773

9 GenAI Usage Disclosure

Generative AI tools were used as writing and coding assistants. Specifically, they supported language refinement, code checking, and debugging. All scientific contributions, experiments, results, and conclusions were independently verified by the authors, who take full responsibility for the final content of the manuscript.

References

- [1] Ziyad AlGhamdi, Fuad Jamour, Spiros Skiadopoulos, and Panos Kalnis. 2017. A Benchmark for Betweenness Centrality Approximation Algorithms on Large Graphs. In *Proceedings of the 29th International Conference on Scientific and Statistical Database Management* (Chicago, IL, USA) (SSDBM '17). Association for Computing Machinery, New York, NY, USA, Article 6, 12 pages. doi:10.1145/3085504.3085510
- [2] Albert-László Barabási and Réka Albert. 1999. Emergence of scaling in random networks. *Science* 286, 5439 (1999), 509–512.
- [3] Massimo Bernaschi, Giancarlo Carbone, and Flavio Vella. 2016. Scalable betweenness centrality on multi-GPU systems. In *Proceedings of the ACM International Conference on Computing Frontiers* (Como, Italy) (CF '16). Association for Computing Machinery, New York, NY, USA, 29–36. doi:10.1145/2903150.2903153
- [4] Béla Bollobás, Christian Borgs, Jennifer Chayes, and Oliver Riordan. 2003. Directed Scale-Free Graphs. In *Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms* (SODA '03). Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 132–139.
- [5] Michele Borassi, Pierluigi Crescenzi, and Michel Habib. 2016. Into the Square: On the Complexity of Some Quadratic-time Solvable Problems. *Electronic Notes in Theoretical Computer Science* 322 (2016), 51–67. doi:10.1016/j.entcs.2016.03.005
- [6] Michele Borassi and Emanuele Natale. 2019. KADABRA is an Adaptive Algorithm for Betweenness via Random Approximation. *ACM J. Exp. Algorithmics* 24, Article 1.2 (Feb. 2019), 35 pages. doi:10.1145/3284359
- [7] Ulrik Brandes. 2001. A faster algorithm for betweenness centrality. *The Journal of Mathematical Sociology* 25, 2 (2001), 163–177. doi:10.1080/0022250X.2001.9990249
- [8] Ulrik Brandes and Christian Pich. 2007. Centrality estimation in large networks. *International Journal of Bifurcation and Chaos* 17, 07 (2007), 2303–2318.
- [9] Filippo Brunelli, Pierluigi Crescenzi, and Laurent Viennot. 2024. Making Temporal Betweenness Computation Faster and Restless. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Barcelona, Spain) (KDD '24). Association for Computing Machinery, New York, NY, USA, 163–174. doi:10.1145/3637528.3671825
- [10] Wei Chen, Tie-yan Liu, Yanyan Lan, Zhi-ming Ma, and Hang Li. 2009. Ranking Measures and Loss Functions in Learning to Rank. In *Advances in Neural Information Processing Systems*, Y. Bengio, D. Schuurmans, J. Lafferty, C. Williams, and A. Culotta (Eds.), Vol. 22. Curran Associates, Inc., Red Hook, NY, USA. https://proceedings.neurips.cc/paper_files/paper/2009/file/2f55707d4193dc27118a0f19a1985716-Paper.pdf
- [11] Cyrus Cousins, Chloe Wohlgemuth, and Matteo Riondato. 2023. Bavarian: Betweenness Centrality Approximation with Variance-Aware Rademacher Averages. *ACM Trans. Knowl. Discov. Data* 17, 6, Article 78 (March 2023), 47 pages. doi:10.1145/3577021
- [12] Changjun Fan, Li Zeng, Yuhui Ding, Muhao Chen, Yizhou Sun, and Zhong Liu. 2019. Learning to Identify High Betweenness Centrality Nodes from Scratch: A Novel Graph Neural Network Approach. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management* (Beijing, China) (CIKM '19). Association for Computing Machinery, New York, NY, USA, 559–568. doi:10.1145/3357384.3357799
- [13] Linton C. Freeman. 1977. A Set of Measures of Centrality Based on Betweenness. *Sociometry* 40, 1 (March 1977), 35. doi:10.2307/3033543
- [14] Tobias Friedrich and Anton Krohmer. 2018. On the Diameter of Hyperbolic Random Graphs. *SIAM Journal on Discrete Mathematics* 32, 2 (2018), 1314–1334. arXiv:https://doi.org/10.1137/17M1123961 doi:10.1137/17M1123961
- [15] Robert Geisberger, Peter Sanders, and Dominik Schultes. 2008. Better Approximation of Betweenness Centrality. In *2008 Proceedings of the Workshop on Algorithm Engineering and Experiments (ALENEX)*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 90–100. doi:10.1137/1.9781611972887.9
- [16] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable Feature Learning for Networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (San Francisco, California, USA) (KDD '16). Association for Computing Machinery, New York, NY, USA, 855–864. doi:10.1145/2939672.2939754
- [17] Luca Gugelmann, Konstantinos Panagiotou, and Ueli Peter. 2012. Random Hyperbolic Graphs: Degree Sequence and Clustering. In *Automata, Languages, and Programming*, Artur Czumaj, Kurt Mehlhorn, Andrew Pitts, and Roger Wattenhofer (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 573–585.
- [18] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. 2008. Exploring Network Structure, Dynamics, and Function using NetworkX. In *Proceedings of the 7th Python in Science Conference*, Gaël Varoquaux, Travis Vaught, and Jarrod Millman (Eds.). SciPy, Pasadena, CA, USA, 11–15. doi:10.25080/tcww9851
- [19] Franziska Heeg and Ingo Scholtes. 2024. Using time-aware graph neural networks to predict temporal centralities in dynamic graphs. In *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '24). Curran Associates Inc., Red Hook, NY, USA, Article 949, 30 pages.
- [20] Loc Hoang, Matteo Pontecorvi, Roshan Dathathri, Gurbinder Gill, Bozhi You, Keshav Pingali, and Vijaya Ramachandran. 2019. A round-efficient distributed betweenness centrality algorithm. In *Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming* (Washington, District of Columbia) (PPoPP '19). Association for Computing Machinery, New York, NY, USA, 272–286. doi:10.1145/3293883.3295729
- [21] Qiang-Sheng Hua, Haoqiang Fan, Ming Ai, Lixiang Qian, Yangyang Li, Xuanhua Shi, and Hai Jin. 2016. Nearly Optimal Distributed Algorithm for Computing Betweenness Centrality. In *2016 IEEE 36th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, Piscataway, NJ, USA, 271–280. doi:10.1109/ICDCS.2016.89
- [22] Russell Impagliazzo, Ramamohan Paturi, and Francis Zane. 2001. Which Problems Have Strongly Exponential Complexity? *J. Comput. System Sci.* 63, 4 (2001), 512–530. doi:10.1006/jcss.2001.1774
- [23] IA Kasyanov, Pim van der Hoorn, D Kriukov, and MV Tamm. 2023. Nearest-neighbor directed random hyperbolic graphs. *Physical Review E* 108, 5 (2023), 054310.
- [24] Dmitri Kriukov, Fragkiskos Papadopoulos, Maksim Kitsak, Amin Vahdat, and Marián Boguñá. 2010. Hyperbolic geometry of complex networks. *Phys. Rev. E* 82 (Sep 2010), 036106. Issue 3. doi:10.1103/PhysRevE.82.036106
- [25] Ranjan Kumar Behera, Santanu Kumar Rath, Sanjay Misra, Robertas Damaševičius, and Rytis Maskeliūnas. 2019. Distributed Centrality Analysis of Social Network Data Using MapReduce. *Algorithms* 12, 8 (2019), 161. doi:10.3390/a12080161
- [26] J. Lee, Y. Lee, S. M. Oh, and B. Kahng. 2021. Betweenness centrality of teams in social networks. *Chaos: An Interdisciplinary Journal of Nonlinear Science* 31, 6 (2021), 061108. doi:10.1063/1.5054127
- [27] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>.
- [28] Cong Li, Qian Li, Piet Miegheem, H. Stanley, and Huijuan Wang. 2014. Correlation between centrality metrics and their application to the opinion model. *The European Physical Journal B* 88 (09 2014). doi:10.1140/epjb/e2015-50671-y
- [29] Ahmad Mahmoody, Charalampos E. Tsourakakis, and Eli Upfal. 2016. Scalable Betweenness Centrality Maximization via Sampling. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (San Francisco, California, USA) (KDD '16). Association for Computing Machinery, New York, NY, USA, 1765–1773. doi:10.1145/2939672.2939869
- [30] Sunil Kumar Maurya, Xin Liu, and Tsuyoshi Murata. 2019. Fast Approximations of Betweenness Centrality with Graph Neural Networks. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management* (Beijing, China) (CIKM '19). Association for Computing Machinery, New York, NY, USA, 2149–2152. doi:10.1145/3357384.3358080
- [31] Sunil Kumar Maurya, Xin Liu, and Tsuyoshi Murata. 2021. Graph Neural Networks for Fast Node Ranking Approximation. *ACM Trans. Knowl. Discov. Data* 15, 5, Article 78 (May 2021), 32 pages. doi:10.1145/3446217
- [32] Martin Mirakyan. 2021. ABCDE: Approximating Betweenness-Centrality ranking with progressive-DropEdge. *PeerJ Computer Science* 7 (2021), e699.
- [33] Tobias Müller and Merlijn Staps. 2019. The diameter of KPKVB random graphs. *Advances in Applied Probability* 51, 2 (2019), 358–377. <http://www.jstor.org/stable/45277962>
- [34] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. arXiv preprint arXiv:1912.01703. doi:10.48550/arXiv.1912.01703
- [35] Leonardo Pellegrina and Fabio Vandin. 2023. SILVAN: Estimating Betweenness Centralities with Progressive Sampling and Non-uniform Rademacher Bounds. *ACM Trans. Knowl. Discov. Data* 18, 3, Article 52 (Dec. 2023), 55 pages. doi:10.1145/3628601
- [36] Matteo Riondato and Evgenios M. Kornaropoulos. 2014. Fast approximation of betweenness centrality through sampling. In *Proceedings of the 7th ACM International Conference on Web Search and Data Mining* (New York, New York, USA) (WSDM '14). Association for Computing Machinery, New York, NY, USA, 413–422. doi:10.1145/2556195.2556224
- [37] Matteo Riondato and Eli Upfal. 2018. ABRA: Approximating Betweenness Centrality in Static and Dynamic Graphs with Rademacher Averages. *ACM Trans. Knowl. Discov. Data* 12, 5, Article 61 (July 2018), 38 pages. doi:10.1145/3208351
- [38] Ryan A. Rossi and Nesreen K. Ahmed. 2015. The Network Data Repository with Interactive Graph Analytics and Visualization. In *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*. AAAI Press, Palo Alto, CA, USA, 4292–4293. doi:10.1609/aaai.v29i1.9277
- [39] Edgar Solomonik, Maciej Besta, Flavio Vella, and Torsten Hoefer. 2017. Scaling betweenness centrality using communication-efficient sparse matrix multiplication. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Denver, Colorado) (SC '17). Association for Computing Machinery, New York, NY, USA, 14 pages. doi:10.1145/3126908.3126971

- [40] Christian L. Staudt, Aleksejs Sazonovs, and Henning Meyerhenke. 2015. NetworKit: A Tool Suite for Large-scale Complex Network Analysis. arXiv preprint arXiv:1403.3005. doi:10.48550/arXiv.1403.3005
- [41] Flavio Vella, Massimo Bernaschi, and Giancarlo Carbone. 2018. Dynamic Merging of Frontiers for Accelerating the Evaluation of Betweenness Centrality. *ACM J. Exp. Algorithmics* 23, Article 1.4 (March 2018), 19 pages. doi:10.1145/3182656
- [42] Moritz von Looz and Henning Meyerhenke. 2016. Querying Probabilistic Neighborhoods in Spatial Data Sets Efficiently. In *Combinatorial Algorithms*, Veli Mäkinen, Simon J. Puglisi, and Leena Salmela (Eds.). Springer International Publishing, Cham, 449–460.
- [43] Yuichi Yoshida. 2014. Almost linear-time algorithms for adaptive betweenness centrality using hypergraph sketches. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (New York, New York, USA) (*KDD '14*). Association for Computing Machinery, New York, NY, USA, 1416–1425. doi:10.1145/2623330.2623626
- [44] Shiqi Zhang, Renchi Yang, Jing Tang, Xiaokui Xiao, and Bo Tang. 2023. Efficient Approximation Algorithms for Spanning Centrality. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23)*. Association for Computing Machinery, New York, NY, USA, 3386–3395. doi:10.1145/3580305.3599323